

Collaborative Filtering

Recommendation System

Hoa Vu
CS 663

SDSU

April 15, 2026

The Recommendation System

Goal: Predict what items a user will like, based on past behavior.

- ▶ Netflix: recommend movies.
- ▶ Amazon: recommend products.
- ▶ Spotify: recommend songs.

Key insight: Users who agreed in the past tend to agree in the future.

User–Item Rating Matrix

Store ratings in an $m \times n$ matrix R (m users, n items).

	Item 1	Item 2	Item 3	Item 4	Item 5
User 1	5	3	?	1	?
User 2	4	?	4	1	2
User 3	1	1	?	5	4
User 4	?	?	3	?	5

- ▶ “?” means the user has not rated the item.
- ▶ **Task:** fill in the missing entries.

Sparsity

In practice the matrix is extremely sparse.

- ▶ Netflix (2006): 480,000 users, 17,770 movies, only 1.2% filled in.
- ▶ Amazon: millions of products, most users rate fewer than 10 items.

Challenge: extrapolate preferences from very few observations.

Two Flavors

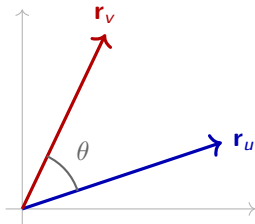
- ▶ **User–User CF**: find users similar to the target user; borrow their ratings.
- ▶ **Item–Item CF**: find items similar to those the target user liked; recommend those.

Both rely on a **similarity measure** between rows (users) or columns (items) of R .

Cosine Similarity

Treat each user's ratings as a vector $\mathbf{r}_u \in \mathbb{R}^n$ (0 for unrated).

$$\text{sim}_{\cos}(u, v) = \frac{\mathbf{r}_u \cdot \mathbf{r}_v}{\|\mathbf{r}_u\| \|\mathbf{r}_v\|} = \cos \theta$$



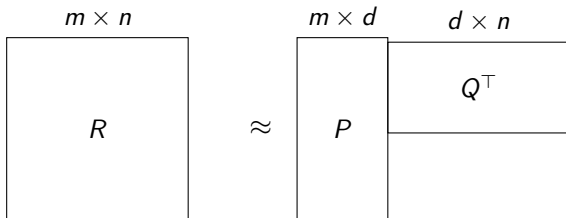
- ▶ $\cos \theta \approx 1$: users rate similarly.
- ▶ $\cos \theta \approx 0$: orthogonal preferences.
- ▶ $\cos \theta < 0$: opposite tastes.

Latent Factor Models

Idea: approximate $R \approx PQ^T$ where

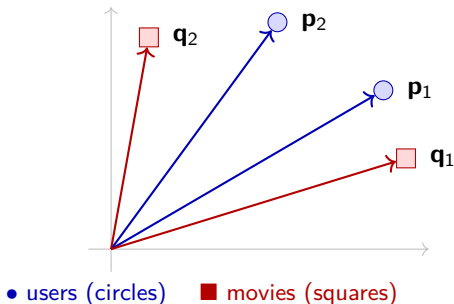
$$P \in \mathbb{R}^{m \times d}, \quad Q \in \mathbb{R}^{n \times d}, \quad d \ll \min(m, n).$$

- ▶ Each row $\mathbf{p}_u \in \mathbb{R}^d$: user's **latent preferences**.
- ▶ Each row $\mathbf{q}_i \in \mathbb{R}^d$: item's **latent attributes**.
- ▶ Predicted rating: $\hat{r}_{ui} = \mathbf{p}_u \cdot \mathbf{q}_i$.



Intuition: Embedding Users and Movies

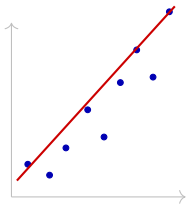
Each user and movie is mapped to a point in the **same** d -dimensional space.



- ▶ $\hat{r}_{ij} = \mathbf{p}_i \cdot \mathbf{q}_j$: high when vectors point in the **same direction**.
- ▶ User i is predicted to like movie j iff their embeddings are **close in angle**.

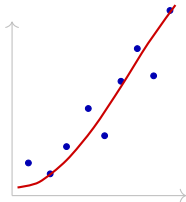
Underfitting, Overfitting, and Just Right

Underfitting



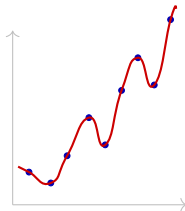
high bias, low variance

Just Right



low bias, low variance

Overfitting



low bias, high variance

What Are Bias and Variance?

Imagine training on many different datasets drawn from the same distribution.

Bias — how far off the average prediction is from the truth

- ▶ High bias: model is too simple to capture the true pattern

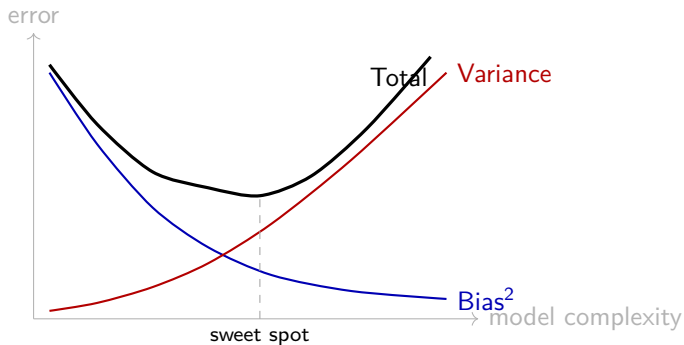
Variance — how much the prediction changes across datasets

- ▶ High variance: model is so flexible it chases noise

Key tension: increasing model complexity reduces bias but raises variance.

Bias–Variance Tradeoff

$$\mathbb{E}[\text{error}] = \underbrace{\text{Bias}^2}_{\text{underfitting}} + \underbrace{\text{Variance}}_{\text{overfitting}} + \sigma_{\text{noise}}^2$$



Regularization

A polynomial $y = c_d x^d + \dots + c_1 x + c_0$ is likely to **overfit** for large d .

One Solution: L_2 regularization: penalize large coefficients by adding to the loss:

$$\mathcal{L} = \underbrace{\sum_i (y_i - \hat{y}_i)^2}_{\text{fit}} + \lambda \underbrace{\sum_{k=0}^d c_k^2}_{\text{penalty}}$$

- ▶ $\lambda > 0$ controls the trade-off between fit and complexity.
- ▶ Large λ : shrinks coefficients toward zero $\Rightarrow$ simpler model.
- ▶ Small λ : nearly unconstrained $\Rightarrow$ risk of overfitting.

Regularization

A polynomial $y = c_d x^d + \dots + c_1 x + c_0$ is likely to **overfit** for large d .

One Solution: L_2 **regularization**: penalize large coefficients by adding to the loss:

$$\mathcal{L} = \underbrace{\sum_i (y_i - \hat{y}_i)^2}_{\text{fit}} + \lambda \underbrace{\sum_{k=0}^d c_k^2}_{\text{penalty}}$$

- ▶ $\lambda > 0$ controls the trade-off between fit and complexity.
- ▶ Large λ : shrinks coefficients toward zero $\Rightarrow$ simpler model.
- ▶ Small λ : nearly unconstrained $\Rightarrow$ risk of overfitting.

L_1 Regularization (Lasso)

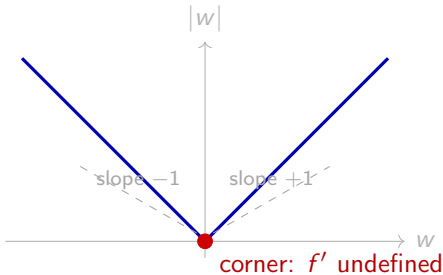
Another solution: penalize the *sum of absolute values* of coefficients:

$$\mathcal{L} = \underbrace{\sum_i (y_i - \hat{y}_i)^2}_{\text{fit}} + \lambda \underbrace{\sum_{k=0}^d |c_k|}_{\text{penalty}}$$

- ▶ Tends to produce **sparse** solutions: many coefficients driven to exactly zero.
- ▶ Effectively performs **feature selection**.
- ▶ Non-differentiable at $c_k = 0$: requires subgradient or coordinate-descent methods.

Why L_1 Breaks Gradient Descent

The function $f(w) = |w|$ has a **corner** at $w = 0$. The derivative does not exist there.



Standard gradient descent requires $f'(w)$ at every step — undefined at $w = 0$.

Subgradient: The Idea

Key insight: at a non-differentiable point, replace the gradient with any value from the *valid slope range*.

For $f(w) = |w|$, the **subgradient** g is:

$$g \in \begin{cases} \{+1\} & w > 0 \\ \{-1\} & w < 0 \\ [-1, +1] & w = 0 \end{cases}$$

- ▶ Where f is differentiable, the subgradient is just the ordinary gradient.
- ▶ At the corner, *any* slope between -1 and $+1$ is a valid choice.
- ▶ Common choice at $w = 0$: pick $g = 0$ (stays put if already there).

Subgradient Descent

Same update rule as gradient descent, but use a subgradient g in place of the gradient:

$$w \leftarrow w - \eta g, \quad g \in \partial f(w)$$

- ▶ $\partial f(w)$ denotes the set of all subgradients at w (the **subdifferential**).
- ▶ Convergence is **guaranteed** for convex f with a diminishing step size.
- ▶ Typically slower than gradient descent on smooth functions.

For L_1 regularization, at each step apply:

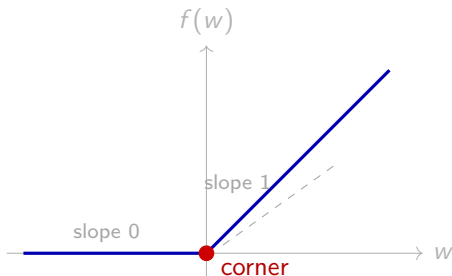
$$c_k \leftarrow c_k - \eta \lambda g_k, \quad g_k = \begin{cases} +1 & c_k > 0 \\ -1 & c_k < 0 \\ 0 & c_k = 0 \end{cases}$$

This gradually drives small coefficients to **exactly zero**.

Example: Subgradient of ReLU

The ReLU function $f(w) = \max(0, w)$ is non-differentiable at $w = 0$.
The subgradient of ReLU is:

$$g \in \begin{cases} \{1\} & w > 0 \\ \{0\} & w < 0 \\ [0, 1] & w = 0 \end{cases}$$



- ▶ $w > 0$: ordinary derivative = 1.
- ▶ $w < 0$: ordinary derivative = 0.
- ▶ $w = 0$: any value in $[0, 1]$.
- ▶ Most frameworks use $g = 0$ at $w = 0$.

Objective Function

Minimize the regularized squared error over observed ratings:

$$\mathcal{L}(P, Q) = \sum_{(u,i): r_{ui} \text{ known}} (r_{ui} - \mathbf{p}_u \cdot \mathbf{q}_i)^2 + \lambda(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2)$$

- ▶ First term: fit observed ratings.
- ▶ Second term: L_2 regularization to prevent overfitting.
- ▶ $\lambda > 0$: regularization strength.

Gradient Derivation

Focus on a single term. Let $e_{ui} = r_{ui} - \mathbf{p}_u \cdot \mathbf{q}_i$. The per-pair loss is:

$$\ell_{ui} = e_{ui}^2 + \lambda(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2)$$

Gradient with respect to $\mathbf{p}_u$:

$$\frac{\partial \ell_{ui}}{\partial \mathbf{p}_u} = 2e_{ui} \cdot \frac{\partial e_{ui}}{\partial \mathbf{p}_u} + 2\lambda \mathbf{p}_u = -2e_{ui} \mathbf{q}_i + 2\lambda \mathbf{p}_u$$

Gradient with respect to $\mathbf{q}_i$ (symmetric):

$$\frac{\partial \ell_{ui}}{\partial \mathbf{q}_i} = -2e_{ui} \mathbf{p}_u + 2\lambda \mathbf{q}_i$$

Dropping the constant 2, the SGD updates are:

$$\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta(e_{ui} \mathbf{q}_i - \lambda \mathbf{p}_u), \quad \mathbf{q}_i \leftarrow \mathbf{q}_i + \eta(e_{ui} \mathbf{p}_u - \lambda \mathbf{q}_i)$$

Stochastic Gradient Descent

For each observed rating r_{ui} , compute the error:

$$e_{ui} = r_{ui} - \mathbf{p}_u \cdot \mathbf{q}_i$$

Update rules (learning rate η):

$$\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta (e_{ui} \mathbf{q}_i - \lambda \mathbf{p}_u)$$

$$\mathbf{q}_i \leftarrow \mathbf{q}_i + \eta (e_{ui} \mathbf{p}_u - \lambda \mathbf{q}_i)$$

Iterate over all known ratings until convergence.

Stochastic Gradient Descent

Observation: $\mathcal{L}$ is not jointly convex in P and Q , but gradient descent still finds good local minima in practice.

Algorithm: stochastic gradient descent (SGD): update on one observed rating (u, i) at a time:

$$\mathbf{p}_u \leftarrow \mathbf{p}_u + \eta (e_{ui} \mathbf{q}_i - \lambda \mathbf{p}_u), \quad \mathbf{q}_i \leftarrow \mathbf{q}_i + \eta (e_{ui} \mathbf{p}_u - \lambda \mathbf{q}_i)$$

where $e_{ui} = r_{ui} - \mathbf{p}_u \cdot \mathbf{q}_i$.

Incorporating Biases

Some users always rate high; some items are universally loved.

Extended model:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u \cdot \mathbf{q}_i$$

- ▶ μ : global average rating.
- ▶ b_u : user bias (lenient vs. strict).
- ▶ b_i : item bias (popular vs. niche).
- ▶ $\mathbf{p}_u \cdot \mathbf{q}_i$: user–item interaction term.

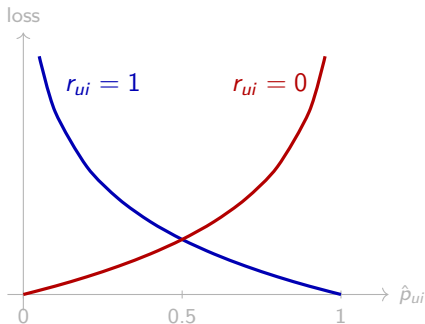
The biases b_u, b_i are learned jointly with P and Q .

Cross Entropy Loss

When ratings are **binary** (liked / not liked). Replace squared error with **binary cross-entropy**:

$$\mathcal{L} = - \sum_{(u,i) \text{ observed}} \left[r_{ui} \log \hat{p}_{ui} + (1 - r_{ui}) \log(1 - \hat{p}_{ui}) \right]$$

Cross Entropy Loss: Shape



When $r_{ui} = 1$ (liked):

- ▶ Loss = $-\log \hat{p}_{ui}$
- ▶ Zero only when $\hat{p}_{ui} \rightarrow 1$
- ▶ Penalizes low confidence

When $r_{ui} = 0$ (not liked):

- ▶ Loss = $-\log(1 - \hat{p}_{ui})$
- ▶ Zero only when $\hat{p}_{ui} \rightarrow 0$
- ▶ Penalizes high confidence

Both curves $\rightarrow \infty$ for confidently wrong predictions.